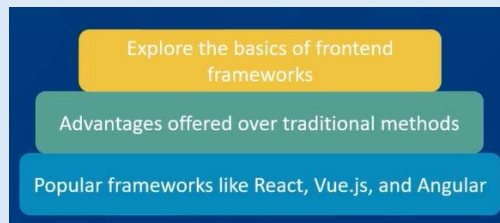


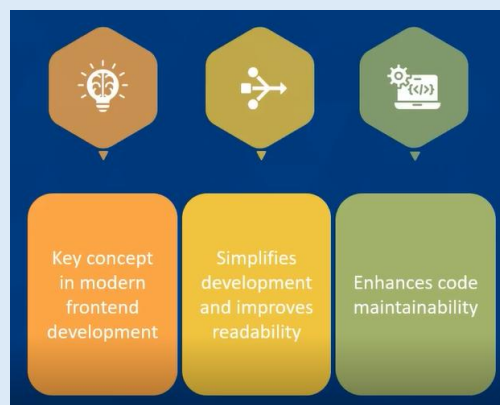
Module 7: Frontend Frameworks

Module Overview

- Frontend frameworks like React, Angular, and Vue.js help create dynamic and efficient user interfaces, offering clear advantages over traditional static methods.



- Modern development has shifted from rigid, page-based designs to flexible, component-based architecture, allowing code reuse and easier maintenance.
- Components act as self-contained building blocks, making it simpler to manage complex interfaces and promote consistent design across an application.
- Declarative programming enables developers to describe what the UI should look like rather than detailing step-by-step instructions, improving clarity and reducing errors.



- Single Page Applications (SPAs) load content dynamically without full page reloads, resulting in smoother, faster, and more seamless user experiences.

Introduction to Frontend Frameworks

- Frontend frameworks provide a structured approach to building user interfaces, promoting efficiency, consistency, and maintainability in web development.



- Framework selection should be based on project requirements, team expertise, performance needs and the strength of community support and documentation.



- Popular frameworks like React, Angular and Vue.js offer component-based architecture, state management, routing and tooling to streamline development.

```
// Simple React component
import React from 'react';

function HelloWorld() {
  return <h1>Hello, World!</h1>;
}

export default HelloWorld;
```

```
// Simple Angular component
import { Component } from '@angular/core';

@Component({
  selector: 'app-hello-world',
  template: '<h1>Hello, World!</h1>',
})
export class HelloWorldComponent {}
```

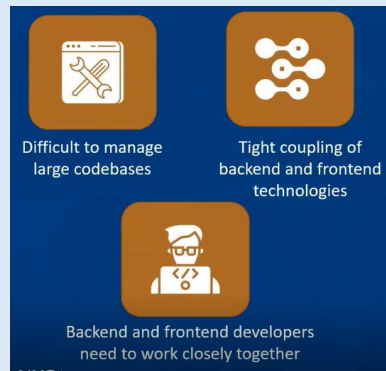
```
// Simple Vue.js component
<div id="app">
  {{ message }}
</div>
```

```
new Vue({
  el: '#app',
  data: {
    message: 'Hello Vue.js!'
  }
})
```

- Pre-built, reusable components and optimised rendering help improve productivity, performance and user experience in single-page applications.
- Key differences among frameworks include React's virtual DOM and JSX, Angular's full-featured TypeScript support, and Vue's simplicity and flexibility.

Traditional Approach to Frontend Development

- Traditional frontend development relies heavily on server-side rendering, where the server generates and delivers complete HTML pages to the client.
- HTML, CSS and JavaScript form the foundation of frontend development, but in the traditional model, dynamic interactions are manually scripted.
- Tight coupling between frontend and backend code makes maintenance difficult and slows development due to limited separation of concerns.



- Limited reusability of code and full page reloads contribute to inconsistent design, poor performance, and a less responsive user experience.



- Scalability challenges emerge as applications grow, with increased server load and latency impacting overall performance and efficiency.



Modern Approach to Frontend Development

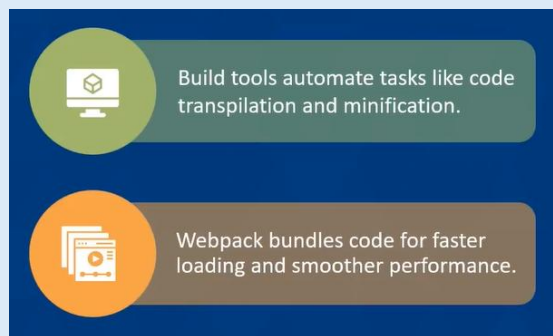
- Reactive and declarative programming simplifies UI updates by automatically syncing changes between data and the user interface.

The declarative approach defines behaviour, allowing the framework to update the UI.

```
// React State Example
const [count, setCount] = useState(0);

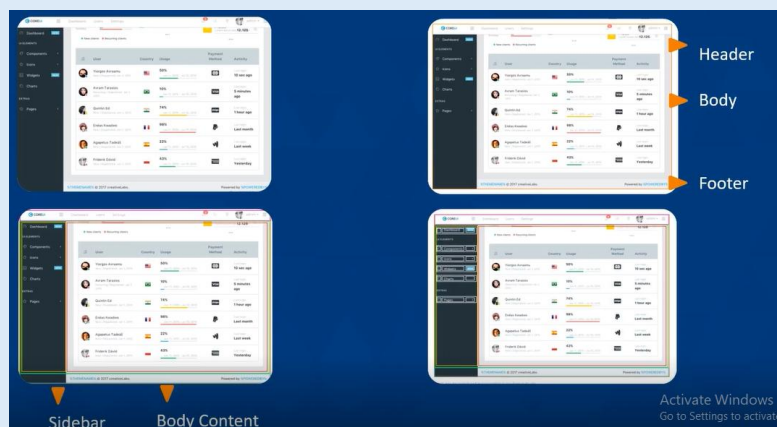
return (
  <div>
    <p>{count}</p>
    <button onClick={() => setCount(count + 1)}>Increment</button>
  </div>
);
```

- Modern frontend development leverages JavaScript frameworks to enable client-side rendering and component-based architecture for dynamic web applications.
- Applications load faster during navigation due to reduced server requests, improving user experience and lowering server load and data usage.
- Components encapsulate structure, style and behaviour, supporting modular design, code reuse and easier application maintenance and scaling.
- Built tools and module bundlers streamline development by automating tasks and optimising application performance during runtime.



Component-Based Design

- Components-based design structures web applications into reusable, self-contained units that serve as the fundamental building blocks of the user interface.
- Each component can contain other nested components, forming a hierarchical structure that simplifies independent development and maintenance of application parts.



- React uses JSX syntax, allowing developers to write HTML-like code within JavaScript and apply logic before rendering elements.

- Component hierarchies should follow a clear parent-child relationship, promoting separation of concern and single-responsibility design principles.



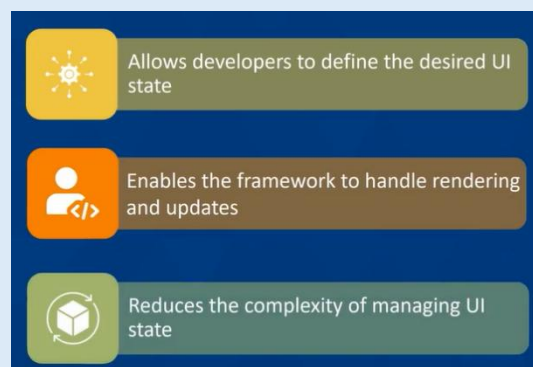
- Adopting best practices such as small, focused components and consistent naming enhances code readability, maintainability and scalability in modern web development.

Declarative Approach to Development

- Declarative development eliminates manual DOM manipulation by allowing developers to specify the desired UI state based on data.

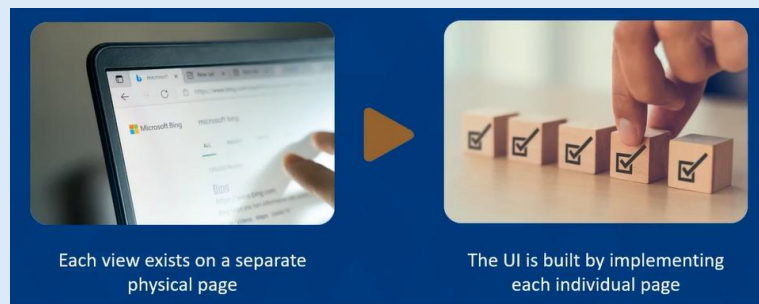
The diagram shows two code snippets on the left and a UI preview on the right. The first snippet is HTML: `<button onclick="addItem()">Add</button>` and `<ul id="list"></ul>`. The second snippet is JavaScript: `const listElement = document.querySelector("#list");`, `let count = 0;`, and a function `addItem()` that appends a new list item. The UI preview on the right shows a button labeled 'Add' and a list containing 'item 1', 'item 2', and 'item 3'.

- Traditional imperative approaches require developers to handle every UI update, which becomes error-prone as applications grow in complexity.
- Modern frameworks like React shift rendering responsibilities to the framework, ensuring UI updates automatically when data changes.
- This approach improves maintainability, reduces bugs, and enables developers to focus on business logic rather than UI rendering mechanics.
- Declarative programming supports scalability in large applications by simplifying complex UI updates and enabling more modular development practices.

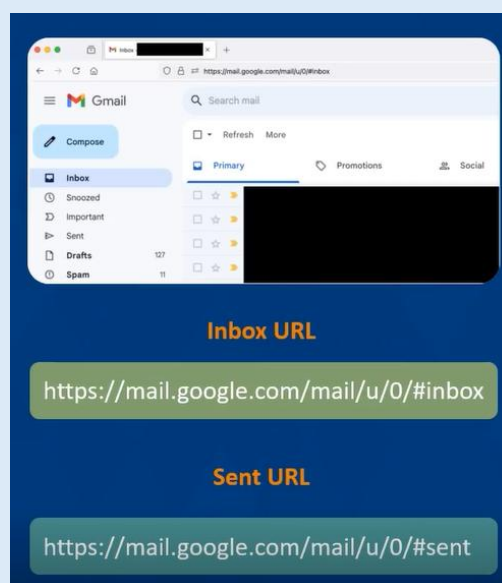


Single-Page Applications

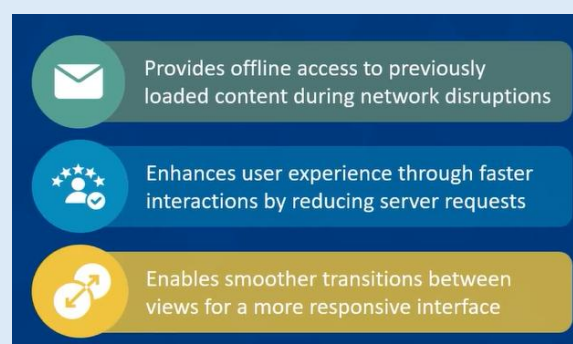
- Single page applications load a single HTML file and dynamically update content without requiring full-page reloads during user interactions.



- JavaScript handles client-side routing and view updates, enabling asynchronous data fetching and a seamless navigation experience.



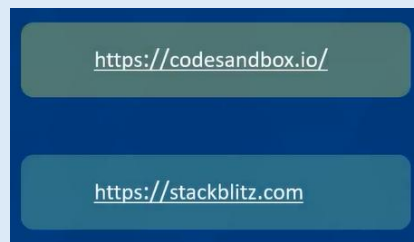
- SPAs improve user experience by reducing server requests, allowing faster transitions and supporting limited offline capabilities.



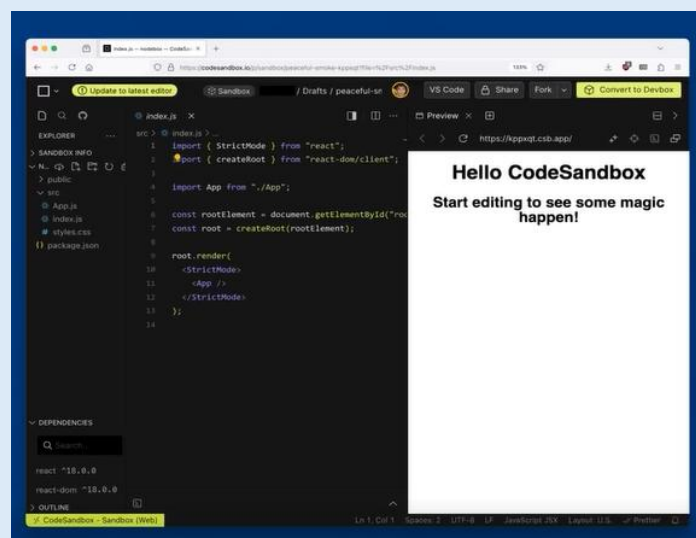
- Key features include dynamic UI updates, asynchronous content loading and client-side routing to simulate multi-page behaviour within a single page.
- Challenges include longer initial load times, limited SEO compatibility and increased complexity in managing browser compatibility and application state.

Getting Started with Frontend Development Framework

- Cloud-based development environments like CodeSandbox and StackBlitz offer a quick and setup-free way to begin frontend development.



- These platforms simplify the learning process by allowing users to code immediately without dealing with local configuration issues.
- CodeSandbox provides an in-browser editor and live preview, enabling real-time feedback while working on applications like React.



- Setting up a local environment typically involves installing Node.js, which includes the Node Package Manager for managing dependencies.



- Visual Studio Code is a widely used code editor offering helpful features and extensions that enhance productivity in frontend development.



Module Summary

- Frontend frameworks like React, Angular and Vue.js enhance efficiency, consistency and maintainability through reusable components and structured design patterns.



- Traditional server-side rendering poses challenges such as poor reusability, performance bottlenecks and limited scalability in modern web applications.
- Modern frontend development relies on client-side rendering, component-based architecture and build tools to deliver dynamic and responsive user experiences.
- Declarative programming and single page applications streamline UI updates and navigation, improving performance and reducing complexity in state management.
- Starting with frontend frameworks is easier using cloud-based environments or local setups with Node.js, NPM and Visual Studio Code, supported by strong developer communities.

THE END